MODULE 09 · DOMAIN 4 - TERRAFORM CONFIGURATION

Complex Types, Expressions and Functions

The type system, the expression forms, and the functions worth recognising by name.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 9 of 27

admin@sthithapragna.com

Terraform's type system exists so that **mistakes surface at plan time** rather than halfway through an apply.

The type system

FAMILY	TYPES
Primitive	string, number, bool
Collection	list, set, map. One element type, many elements
Structural	object, tuple. Each attribute or position typed separately
Special	any accepts anything. null means absent

ON THE EXAM

Collections are homogeneous and structural types are not. That is the line between the two families.

list or set?

LIST

Ordered, duplicates allowed

- Elements have positions, indexed from zero
- The same value may appear twice
- Order is preserved and meaningful
- Reordering it is a real change

SET

Unordered, no duplicates

- Elements have no position at all
- Duplicates collapse into one
- Order is not preserved or meaningful
- Reordering it is not a change

THE TIEBREAKER

This is the most consequential distinction in the type system, because count uses positions and for_each uses keys. Module ten.

map or object?

MAP

Any keys, one value type

- Keys are arbitrary strings you choose
- Every value has the same type
- Add keys freely without changing the type
- Good for tags and lookup tables

OBJECT

Fixed keys, each typed

- Attribute names are part of the type
- Each attribute has its own type
- Adding an attribute changes the type
- Good for a structured settings block

THE TIEBREAKER

If callers invent the keys, it is a map. If you fix the keys and they vary in type, it is an object.

Conversion, and null

- Terraform **converts between primitives** where it safely can. The string five satisfies a number constraint.
- It converts between **collection types** too, so a list can become a set, losing order and duplicates.
- **That conversion is lossy and silent.** Converting a list to a set discards ordering with no warning.
- **null** means the value is absent, which is different from an empty string or an empty list.
- Setting an argument to null tells Terraform to **use the provider default**, as if you had omitted it.

> ON THE EXAM
>
> null is not zero, not empty, and not false. It means 'no value here', and providers treat it as omission.

Operators and the conditional expression

- Arithmetic, comparison and logical operators work as you would expect from most languages.
- The **conditional expression** uses a question mark and a colon: condition, then the true value, then the false value.
- Both result values are **converted to the same type**, so returning a string and a number gives you two strings.
- There is **no if statement** in Terraform. The conditional expression is the only branching form.
- Common pattern: choose an instance size or a count based on the environment name.

> ON THE EXAM
>
> Terraform is declarative, so there is no control flow. Conditionals choose between values, they do not choose whether code runs.

for expressions

- A **for expression** transforms one collection into another, in square brackets or braces.
- **Square brackets produce a tuple. Braces produce an object,** and braces need a key and a value.
- An optional **if clause** filters, so you can transform and select at the same time.
- Iterating a map gives you two symbols, the key and the value. Iterating a list gives one.
- This is **not a loop that creates resources**. It builds a value. Resource repetition is count and for_each.

> ON THE EXAM
>
> for expressions build values. count and for_each build resources. Confusing the two is a common wrong answer.

Splat expressions

- A **splat**, written as square bracket star, pulls one attribute from every element of a list.
- It is shorthand for a simple for expression, and far more readable when it fits.
- **It only works over a list.** Applied to a single object it will not do what you expect.
- It pairs naturally with **count**, where instances form a list already.
- With **for_each** the instances form a map, so you want **values()** or a for expression instead.

> ON THE EXAM
>
> Splat suits count. for_each produces a map, and a splat over a map is a common mistake.

String templates and heredocs

- **Interpolation** embeds an expression in a string using a dollar sign and braces.
- **Heredoc** syntax carries multi-line strings. The indented form strips leading whitespace.
- Templates support **directives** with a percent sign and braces, giving conditionals and repetition inside a string.
- To output a literal dollar-brace, **double the dollar sign**. The same trick works for the percent form.
- For anything longer than a line or two, put it in a file and use **templatefile** instead.

> ON THE EXAM
>
> Heredocs are for readability in configuration. Real templates belong in their own file, rendered with templatefile.

Functions

- Terraform ships a **large library of built-in functions**, grouped into documented categories.
- Numeric, string, collection, encoding, filesystem, date and time, hash and crypto, IP network, and type conversion.
- **You cannot write your own function in the Terraform language.** There is no function block in HCL.
- Functions are **pure**. Given the same inputs they return the same output, with no side effects.
- **terraform console** is the right way to learn one. Type the call, see the result, no infrastructure involved.

> ON THE EXAM
>
> Learn the categories rather than memorising signatures. Questions name a task, and you need to know which category solves it.

Provider-defined functions, since Terraform 1.8

- **Providers can now supply functions**, not just resources and data sources. Added in Terraform 1.8.
- Called with a namespaced syntax: **provider, then the local name, then the function name**, separated by double colons.
- Launch examples: **arn_parse and arn_build** in AWS, **direxists** in local, **rfc_3339_parse** in time.
- The provider must be declared in **required_providers**, exactly as in module two.
- They must be **pure and offline**. A provider function makes no API calls and has no side effects.

> ON THE EXAM
>
> The old answer 'Terraform has only built-in functions' is now outdated. You still cannot write one in HCL, but providers can ship them.

Functions worth recognising

FUNCTION	WHAT IT DOES
`lookup(map, key, default)`	Read a map key, falling back to a default if absent
`merge(a, b, ...)`	Combine maps. Later arguments win on conflicts
`toset / tolist / tomap`	Convert between collection types explicitly
`length / keys / values`	Count a collection, or pull out its keys or values
`file / templatefile`	Read a file. templatefile also renders variables into it
`cidrsubnet / cidrhost`	Calculate subnet ranges and host addresses from a prefix

ON THE EXAM

merge and lookup dominate map questions. templatefile is the one people reach for file instead of.

try and can

- **try** evaluates several expressions and returns the first that succeeds without error.
- **can** evaluates one expression and returns true or false, depending on whether it errored.
- Use **try** to supply a fallback for a value that may be absent or the wrong shape.
- Use **can** inside a **validation** condition, where you need a boolean rather than a value. Module twelve.
- **Do not wrap large expressions in try.** It will swallow real errors and hide the cause.

> ON THE EXAM
>
> try returns a value, can returns a boolean. That is the whole difference, and it decides which one a question wants.

Working an example

EXPRESSION	RESULT
`toset(["a", "b", "a"])`	A set of two elements. The duplicate collapses
`length({x = 1, y = 2})`	2. length counts elements of any collection
`merge({a=1, b=2}, {b=9})`	a is 1, b is 9. The later argument wins
`lookup({a=1}, "z", 0)`	0. The key is absent, so the default is returned
`[for s in ["a","b"] : upper(s)]`	A tuple holding A and B
`can(tonumber("abc"))`	false. The conversion errors, so can reports it

> ON THE EXAM
>
> Every one of these can be checked in terraform console in a few seconds. That is the fastest way to make them stick.

Learning this with terraform console

- **terraform console** evaluates expressions interactively against your configuration and state.
- It is the fastest way to settle a question about a function, and it changes nothing.
- It works in an **empty directory** too, if you only want to try functions on literal values.
- **Type an expression, read the result.** Ten minutes here beats memorising signatures.
- It also shows you **types**, so you can see whether something is a list, a set or a tuple.

> ON THE EXAM
>
> This is the one hands-on habit worth building for Domain 4. The type system stops being abstract once you can inspect it.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Adding an item recreated unrelated resources	A list where a set was needed. Positions shifted
The configuration breaks when a key is missing	lookup with a default, or try
We need to parse a provider identifier	A provider-defined function, if the provider ships one
Variables in the template are not being substituted	file was used where templatefile was needed
Choose a size based on the environment	A conditional expression. There is no if statement
Build a list of names from a list of objects	A splat, or a for expression

ON THE EXAM

Most function questions describe a task, not a name. Identify the category first, then the function.

A module creates one storage bucket per team, iterating over a variable declared as a list of strings. Teams are listed alphabetically and there are fourteen of them.

A new team named Atlas is added at the front of the list. The plan proposes to destroy and recreate thirteen existing buckets, in addition to creating one.

What is the cause, and what fixes it?

A The provider requires buckets to be created in order. Add depends_on between them

B Instances are tracked by list position. Use a set of strings with for_each so they are tracked by value

C The list must be sorted with the sort function before use

D Bucket names must be globally unique, so Terraform recreates them when any name changes

◆ PAUSE AND DECIDE

What is the cause, and what fixes it?

A Add depends_on

Ordering is not the problem. Terraform is not confused about sequence; it believes thirteen different buckets are now required.

B Track by value, not position

Correct. A list gives positional identity, so inserting at the front shifts every index. A set with for_each keys instances by value.

C Sort the list first

The list was already alphabetical, and Atlas sorts to the front regardless. Sorting cannot rescue an identity scheme based on position.

D Global uniqueness

True of bucket names and irrelevant here. Only one name was added; the other thirteen are unchanged.

THE SIGNAL

Inserting at the front is the worst case. Appending to the end of a list causes no churn at all, which is why this bug hides for years.

> A module accepts a map of per-environment settings and reads a key called backup_retention_days from it. Three of the four environments define that key. The fourth does not, because backups are handled elsewhere.
>
> The configuration fails for that environment with an error saying the key does not exist.

What is the cleanest fix?

A. Use lookup with a default value for the missing key

B. Change the variable type to any, so the missing key is tolerated

C. Wrap the whole module call in a try expression

D. Add the key to every environment with a value of null

◆ PAUSE AND DECIDE

What is the cleanest fix?

A lookup with a default

Correct. lookup exists for exactly this: read a key, and fall back to a stated default when it is absent. try would also work and is less explicit.

B Type any

Relaxes the type constraint, which was never the problem. The key is genuinely missing, and a looser type does not conjure it.

C try around the module

Exactly the misuse HashiCorp warns about. A large try swallows every error inside it, so a real failure would vanish silently.

D Add the key as null

Workable and spreads the special case into four files. It also asserts a value exists when the point is that it does not.

THE SIGNAL

try and lookup would both work. lookup states the intent in the expression itself, which is why it is the cleaner answer.

> A configuration receives a full AWS resource identifier as a string and needs the account number and the region from it. An engineer has written a chain of split and element calls, with a comment apologising for it.
>
> The team is on Terraform 1.12 and uses a recent version of the AWS provider.

What should be suggested at review?

A Extract the logic into a reusable module so the parsing lives in one place

B Use the provider-defined function provider::aws::arn_parse

C Define a custom function in a locals block and call it where needed

D Use a data source, since parsing requires querying the provider

◆ PAUSE AND DECIDE

What should be suggested at review?

A Extract to a module

Centralises the awkwardness without removing it, and a whole module for a string operation is heavy. Reasonable before 1.8; unnecessary now.

B Use the provider function

Correct. Provider-defined functions arrived in 1.8, and the AWS provider shipped ARN parsing among the first of them.

C Custom function in locals

There is no function block in HCL and a local is a value, not a callable. This half of the old rule still holds.

D Use a data source

Provider functions are pure and offline by design. Parsing a string needs no API call, which is why it is a function and not a data source.

THE SIGNAL

Option C is the answer that used to be closest to right. Providers, not the language, are where Terraform gained extensibility.

A module ships a configuration template containing placeholders for a hostname and a port. It reads the template with the file function and passes the result to a resource.

The deployed configuration contains the literal placeholder text rather than the values. Nothing errors; the file simply arrives unchanged.

What is wrong?

A file reads the file verbatim. templatefile is needed to render values into it

B The template must be renamed with a .tftpl extension before file will render it

C The placeholders must be escaped, because file strips interpolation syntax

D The values are not known at plan time, so they render as placeholders

◆ PAUSE AND DECIDE

What is wrong?

A Use templatefile

Correct. file returns the contents unchanged. templatefile takes a second argument, a map of values, and renders them into the template.

B Rename to .tftpl

That extension is a readable convention that editors recognise. It changes nothing about how a function behaves.

C Escape the placeholders

Backwards. file does not strip anything; it is not looking at the contents at all.

D Unknown at plan time

Would show as (known after apply) in the plan, not as literal placeholder text in a deployed file.

THE SIGNAL

Nothing errors, which is the tell. A wrong function that succeeds is harder to spot than one that fails.

Module 9 recap

- **Collections are homogeneous, structural types are not.** That is the line between the families.
- **list is ordered with duplicates. set is neither.** Positional identity is where resource churn comes from.
- **The conditional expression is the only branching form.** There is no if statement.
- **for expressions build values. count and for_each build resources.**
- **try returns a value, can returns a boolean.** Do not wrap large expressions in try.
- **Provider-defined functions exist since 1.8.** You still cannot write one in HCL.

> ON THE EXAM
>
> null means absent, not empty. Assigning null is the same as omitting the argument entirely.